

Common Concepts for Metaheuristics

Computing optimal solutions is intractable for many optimization problems of industrial and scientific importance. In practice, we are usually satisfied with “good” solutions, which are obtained by heuristic or metaheuristic algorithms. Metaheuristics represent a family of approximate¹ optimization techniques that gained a lot of popularity in the past two decades. They are among the most promising and successful techniques. Metaheuristics provide “acceptable” solutions in a reasonable time for solving hard and complex problems in science and engineering. This explains the significant growth of interest in metaheuristic domain. Unlike exact optimization algorithms, metaheuristics do not guarantee the optimality of the obtained solutions. Instead of approximation algorithms, metaheuristics do not define how close are the obtained solutions from the optimal ones.

The word *heuristic* has its origin in the old Greek word *heuriskein*, which means the art of discovering new strategies (rules) to solve problems. The suffix *meta*, also a Greek word, means “upper level methodology.” The term *metaheuristic* was introduced by F. Glover in the paper [322]. Metaheuristic search methods can be defined as upper level general methodologies (templates) that can be used as guiding strategies in designing underlying heuristics to solve specific optimization problems.

This chapter is organized as follows. Section 1.1 discusses the diverse classical optimization models that can be used to formulate and solve optimization problems. It also introduces the basic concepts of algorithm and problem complexities. Some illustrative easy and hard optimization problems are given. Section 1.2 presents other models for optimization that are not static and deterministic. Those problems are characterized by dynamicity, uncertainty, or multiperiodicity. Then, Section 1.3 outlines the main families of optimization methods: exact versus approximate algorithms, metaheuristic versus approximation algorithms, iterative versus greedy algorithms, single-solution based metaheuristics versus population-based metaheuristics. Finally, an important question one might ask is answered: “when use metaheuristics?” Once the basic material for optimization problems and algorithms are presented, important common concepts of metaheuristics are introduced in Section 1.4, such as the

¹There is a difference between approximate algorithms and approximation algorithms (see Section 1.3.2).

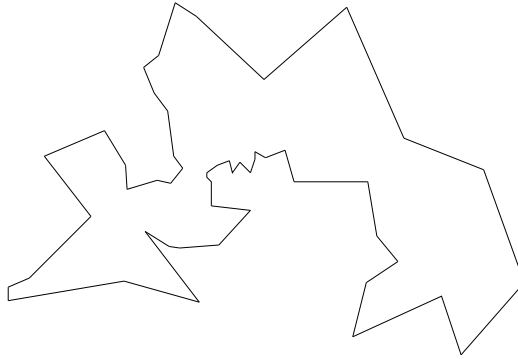


FIGURE 1.4 TSP instance with 52 cities.

The popularity of combinatorial optimization problems stems from the fact that in many real-world problems, the objective function and constraints are of different nature (nonlinear, nonanalytic, black box, etc.) whereas the search space is finite.

Example 1.2 Traveling salesman problem. Perhaps the most popular combinatorial optimization problem is the traveling salesman problem (TSP). It can be formulated as follows: given n cities and a distance matrix $d_{n,n}$, where each element d_{ij} represents the distance between the cities i and j , find a tour that minimizes the total distance. A tour visits each city exactly once (Hamiltonian cycle) (Figs. 1.4 and 1.5). The size of the search space is $n!$ Table 1.2 shows the combinatorial explosion of the number of solutions regarding the number of cities. Unfortunately, enumerating exhaustively all possible solutions is impractical for moderate and large instances.

Another common approach to model decision and optimization problems is *constraint programming* (CP), a programming paradigm that integrates richer modeling tools than the linear expressions of MIP models. A model is composed of a set of variables. Every variable has a finite domain of values. In the model, symbolic and

TABLE 1.2 Effect of the Number of Cities on the Size of the Search Space

Number of Cities n	Size of the Search Space
5	120
10	3, 628, 800
75	2.5×10^{109}



FIGURE 1.5 TSP instance with 24,978 cities.

mathematical constraints related to variables may be expressed. Global constraints represent constraints that refer to a set of variables. Hence, the CP paradigm models the properties of the desired solution. The declarative models in CP are flexible and are in general more compact than in MIP models.

Example 1.3 Assignment problem within constraint programming. The goal is to assign n objects $\{o_1, o_2, \dots, o_n\}$ to n locations $\{l_1, l_2, \dots, l_n\}$ where each object is placed on a different location. Using constraint programming techniques, the model will be the following:

$$\text{all_different}(y_1, y_2, \dots, y_n)$$

where y_i represents the index of the location to which the object o_i is assigned. The global constraint $\text{all_different}(y_1, y_2, \dots, y_n)$ specifies that all variables must be different. If this problem is modeled using an IP model, one has to introduce the following decision variables:

$$x_{ij} = \begin{cases} 1 & \text{if object } o_i \text{ is assigned to location } l_j \\ 0 & \text{otherwise} \end{cases}$$

Hence, much more variables (n^2 instead of n) are declared.

However, it does not mean that solving the problem will be more efficient within constraint programming than using mathematical programming. Solving the problem is another story. The advantage of MIP solvers is that they use relaxations of the problem to prune the search tree, while in CP they use constraint propagation techniques to

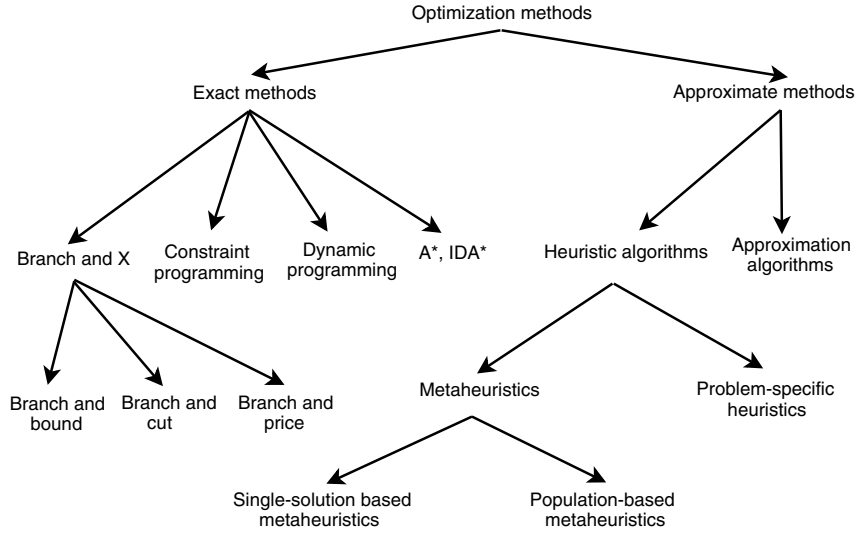


FIGURE 1.7 Classical optimization methods.

integration:

$$f'_{\text{robust}}(x) = \frac{\sum_{i=1}^N f(x + \delta_i)}{N}$$

Robust optimization has to find a trade-off between the quality of solutions and their robustness in terms of decision variable disturbance. This problem may be formulated as a multiobjective optimization problem (see Chapter 4) [415]. Unlike optimization under uncertainty, the objective function in robust optimization is considered as deterministic.

The introduced different variants of optimization models are not exclusive. For instance, many practical optimization problems include uncertainty as well as robustness and/or multiperiodicity. Thus, uncertainty, robustness, and dynamic issues must be jointly considered to solve this class of problems.

1.3 OPTIMIZATION METHODS

Following the complexity of the problem, it may be solved by an exact method or an approximate method (Fig. 1.7). Exact methods¹⁶ obtain optimal solutions and guarantee their optimality. For NP-complete problems, exact algorithms are nonpolynomial-time algorithms (unless $P = NP$). Approximate (or heuristic) methods generate high-quality solutions in a reasonable time for practical use, but there is no guarantee of finding a global optimal solution.

¹⁶In the artificial intelligence community, those algorithms are also named *complete* algorithms.

1.3.1 Exact Methods

In the class of exact methods one can find the following classical algorithms: dynamic programming, branch and X family of algorithms (branch and bound, branch and cut, branch and price) developed in the operations research community, constraint programming, and A* family of search algorithms (A*, IDA*—iterative deepening algorithms) [473] developed in the artificial intelligence community [673]. Those enumerative methods may be viewed as tree search algorithms. The search is carried out over the whole interesting search space, and the problem is solved by subdividing it into simpler problems.

Dynamic programming is based on the recursive division of a problem into simpler subproblems. This procedure is based on the *Bellman's principle* that says that “the subpolicy of an optimal policy is itself optimal” [68]. This stagewise optimization method is the result of a sequence of partial decisions. The procedure avoids a total enumeration of the search space by pruning partial decision sequences that cannot lead to the optimal solution.

The *branch and bound* algorithm and A* are based on an implicit enumeration of all solutions of the considered optimization problem. The search space is explored by dynamically building a tree whose root node represents the problem being solved and its whole associated search space. The leaf nodes are the potential solutions and the internal nodes are subproblems of the total solution space. The pruning of the search tree is based on a bounding function that prunes subtrees that do not contain any optimal solution. A more detailed description of dynamic programming and branch and bound algorithms may be found in Section 5.2.1.

Constraint programming is a language built around concepts of tree search and logical implications. Optimization problems in constraint programming are modeled by means of a set of variables linked by a set of constraints. The variables take their values on a finite domain of integers. The constraints may have mathematical or symbolic forms. A more detailed description of constraint programming techniques may be found in Section 5.3.1.

Exact methods can be applied to small instances of difficult problems. Table 1.4 shows for some popular NP-hard optimization problems the order of magnitude of the maximal size of instances that state-of-the-art exact methods can solve to optimality. Some of the exact algorithms used are implemented on large networks of workstations

TABLE 1.4 Order of Magnitude of the Maximal Size of Instances that State-of-the-Art Exact Methods can Solve to Optimality

Optimization Problems	Quadratic Assignment	Flow-Shop Scheduling (FSP)	Graph Coloring	Capacitated Vehicle Routing
Size of the instances	30 objects	100 jobs 20 machines	100 nodes	60 clients

For some practical problems, this maximum size may be negligible. For the TSP problem, an instance of size 13,509 has been solved to optimality [32].

TABLE 1.5 The Impact of the Structure on the Size of Instances (i.e., Number of Nodes for SOP and GC, Number of Objects for QAP) that State-of-the-Art Exact Methods can Solve to Optimality (SOP: Sequential Ordering Problem; QAP: Quadratic Assignment Problem; GC: Graph Coloring)

Optimization Problem	SOP	QAP	GC
Size of some unsolved instances	53	30	125
Size of some solved instances	70	36	561

(grid computing platforms) composed of more than 2000 processors with more than 2 months of computing time [546]!

The size of the instance is not the unique indicator that describes the difficulty of a problem, but also its structure. For a given problem, some small instances cannot be solved by an exact algorithm while some large instances may be solved exactly by the same algorithm. Table 1.5 shows for some popular optimization problems (e.g., SOP¹⁷: sequential ordering problem; QAP¹⁸: quadratic assignment problem; GC¹⁹: graph coloring) small instances that are not solved exactly and large instances solved exactly by state-of-the-art exact optimization methods.

Example 1.15 Phase transition. In many NP-hard optimization problems, a phase transition occurs in terms of the easiness/hardness of the problem; that is, the difficulty to solve the problem increases until a given size n , and beyond this value the problem is easier to solve [126]. Then, the hardest problems tend to be in the phase transition boundary. Let us consider the number partitioning problem, a widely cited NP-hard problem. Given a bag S of N positive integers $\{a_1, a_2, \dots, a_n\}$, find a partition of the numbers into two equally disjoint bags S_1 and S_2 of cardinality $n/2$ that minimizes the absolute value of the difference of their sums:

$$f = \left| \sum_{i \in S_1} a_i - \sum_{i \in S_2} a_i \right|$$

For the number partitioning problem, the phase transition has been identified around the problem size of $n = 35$ [310,474].

Phase transition phenomena have also been identified in various problems such as graph coloring [126], SAT (propositional satisfiability) [558], CSP (constraint satisfaction problems) [706], traveling salesman problems [312], independent set problems [311], and Hamiltonian circuits [126].

In solving SAT problems, instances before the phase transition are easy to solve and those after the phase transition are mostly unsatisfiable [151]. The phase transition is formulated by the ratio between the number of clauses l and the number

¹⁷See <http://www.iwr.uni-heidelberg.de/groups/comopt/software/TSPLIB95/>.

¹⁸See <http://www.seas.upenn.edu/qaplib/inst.html>.

¹⁹See <http://mat.gsia.cmu.edu/COLOR/instances.html>.

0–1 knapsack problem has an FPTAS with a time complexity of $O(n^3/\epsilon)$. Problems such as the Max-SAT and vertex cover are much harder and are not members of the PTAS class.

The goal in designing an approximation algorithm for a problem is to find tight worst-case bounds. The study of approximation algorithms gives more knowledge on the difficulty of the problem and can help designing efficient heuristics. However, approximation algorithms are *specific* to the target optimization problem (problem dependent). This characteristic limits their applicability. Moreover, in practice, attainable approximations are too far from the global optimal solution, making those algorithms not very useful for many real-life applications.

1.3.3 Metaheuristics

Unlike exact methods, metaheuristics allow to tackle large-size problem instances by delivering satisfactory solutions in a reasonable time. There is no guarantee to find global optimal solutions or even bounded solutions. Metaheuristics have received more and more popularity in the past 20 years. Their use in many applications shows their efficiency and effectiveness to solve large and complex problems. Application of metaheuristics falls into a large number of areas; some of them are

- Engineering design, topology optimization and structural optimization in electronics and VLSI, aerodynamics, fluid dynamics, telecommunications, automotive, and robotics.
- Machine learning and data mining in bioinformatics and computational biology, and finance.
- System modeling, simulation and identification in chemistry, physics, and biology; control, signal, and image processing.
- Planning in routing problems, robot planning, scheduling and production problems, logistics and transportation, supply chain management, environment, and so on.

Optimization is everywhere; optimization problems are often complex; then metaheuristics are everywhere. Even in the research community, the number of sessions, workshops, and conferences dealing with metaheuristics is growing significantly!

Figure 1.8 shows the genealogy of the numerous metaheuristics. The heuristic concept in solving optimization problems was introduced by Polya in 1945 [619]. The simplex algorithm, created by G. Dantzig in 1947, can be seen as a local search algorithm for linear programming problems. J. Edmonds was first to present the greedy heuristic in the combinatorial optimization literature in 1971 [237]. The original references of the following metaheuristics are based on their application to optimization and/or machine learning problems: ACO (ant colonies optimization) [215], AIS (artificial immune systems) [70,253], BC (bee colony) [689,835], CA (cultural algorithms) [652], CEA (coevolutionary algorithms) [375,397], CMA-ES (covariance matrix

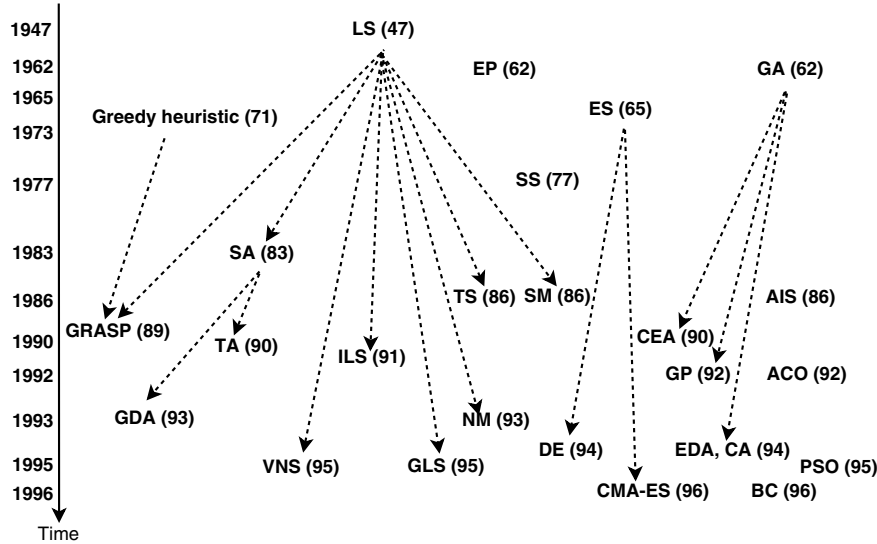


FIGURE 1.8 Genealogy of metaheuristics. The application to optimization and/or machine learning is taken into account as the original date.

adaptation evolution strategy) [363], DE (differential evolution) [626,724], EDA (estimation of distribution algorithms) [47], EP (evolutionary programming) [272], ES (evolution strategies) [642,687], GA (genetic algorithms) [383,384], GDA (great deluge) [229], GLS (guided local search) [805,807], GP (genetic programming) [480], GRASP (greedy adaptive search procedure) [255], ILS (iterated local search) [531], NM (noisy method) [124], PSO (particle swarm optimization) [457], SA (simulated annealing) [114,464], SM (smoothing method) [326], SS (scatter search) [320], TA (threshold accepting) [228], TS (tabu search) [322,364], and VNS (variable neighborhood search) [561].

In designing a metaheuristic, two contradictory criteria must be taken into account: exploration of the search space (diversification) and exploitation of the best solutions found (intensification) (Fig. 1.9). Promising regions are determined by the obtained “good” solutions. In intensification, the promising regions are explored more thoroughly in the hope to find better solutions. In diversification, nonexplored regions

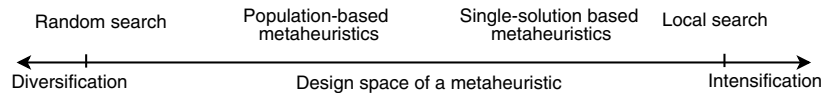


FIGURE 1.9 Two conflicting criteria in designing a metaheuristic: exploration (diversification) versus exploitation (intensification). In general, basic single-solution based metaheuristics are more exploitation oriented whereas basic population-based metaheuristics are more exploration oriented.

must be visited to be sure that all regions of the search space are evenly explored and that the search is not confined to only a reduced number of regions. In this design space, the extreme search algorithms in terms of the exploration (resp. exploitation) are random search (resp. iterative improvement local search). In random search, at each iteration, one generates a random solution in the search space. No search memory is used. In the basic steepest local search algorithm, at each iteration one selects the best neighboring solution that improves the current solution.

Many classification criteria may be used for metaheuristics:

- **Nature inspired versus nonnature inspired:** Many metaheuristics are inspired by natural processes: evolutionary algorithms and artificial immune systems from biology; ants, bees colonies, and particle swarm optimization from swarm intelligence into different species (social sciences); and simulated annealing from physics.
- **Memory usage versus memoryless methods:** Some metaheuristic algorithms are memoryless; that is, no information extracted dynamically is used during the search. Some representatives of this class are local search, GRASP, and simulated annealing. While other metaheuristics use a memory that contains some information extracted online during the search. For instance, short-term and long-term memories in tabu search.
- **Deterministic versus stochastic:** A deterministic metaheuristic solves an optimization problem by making deterministic decisions (e.g., local search, tabu search). In stochastic metaheuristics, some random rules are applied during the search (e.g., simulated annealing, evolutionary algorithms). In deterministic algorithms, using the same initial solution will lead to the same final solution, whereas in stochastic metaheuristics, different final solutions may be obtained from the same initial solution. This characteristic must be taken into account in the performance evaluation of metaheuristic algorithms.
- **Population-based search versus single-solution based search:** Single-solution based algorithms (e.g., local search, simulated annealing) manipulate and transform a single solution during the search while in population-based algorithms (e.g., particle swarm, evolutionary algorithms) a whole population of solutions is evolved. These two families have complementary characteristics: single-solution based metaheuristics are exploitation oriented; they have the power to intensify the search in local regions. Population-based metaheuristics are exploration oriented; they allow a better diversification in the whole search space. In the next chapters of this book, we have mainly used this classification. In fact, the algorithms belonging to each family of metaheuristics share many search mechanisms.
- **Iterative versus greedy:** In iterative algorithms, we start with a complete solution (or population of solutions) and transform it at each iteration using some search operators. Greedy algorithms start from an empty solution, and at each step a decision variable of the problem is assigned until a complete solution is obtained. Most of the metaheuristics are iterative algorithms.

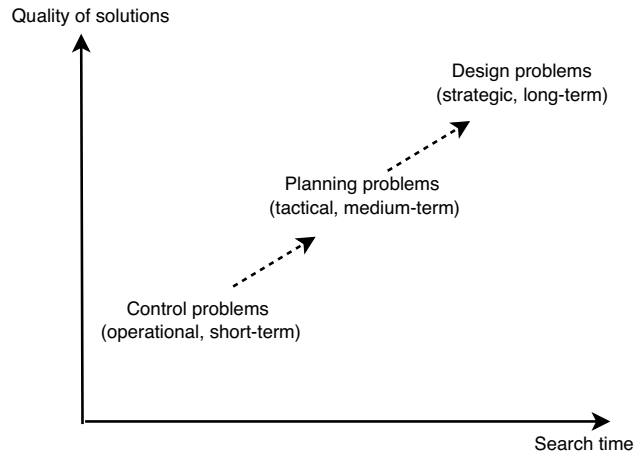


FIGURE 1.15 Different classes of problems in terms of the trade-off between quality of solutions and search time: design (strategic, long-term), planning (tactical, medium-term), control (operational, short-term).

Between these extremes, one can find an intermediate class of problems represented by planning problems and tactical problems (medium-term problems). In this class of problems, a trade-off between the quality of solution and the search time must be optimized. In general, exact optimization algorithms cannot be used to solve such problems.

The *development cost* of solving an optimization problem is also an important issue. Indeed, metaheuristics are easy to design and implement. Open-source and free software frameworks such as ParadisEO allow the efficient design and implementation of metaheuristics for monoobjective and multiobjective optimization problems, hybrid metaheuristics, and parallel metaheuristics. Reusing existing designs and codes will contribute to reducing the development cost.

1.4 MAIN COMMON CONCEPTS FOR METAHEURISTICS

There are two common design questions related to all iterative metaheuristics: the representation of solutions handled by algorithms and the definition of the objective function that will guide the search.

1.4.1 Representation

Designing any iterative metaheuristic needs an encoding (representation) of a solution²⁵. It is a fundamental design question in the development of metaheuristics. The encoding plays a major role in the efficiency and effectiveness of any metaheuristic

²⁵In the evolutionary computation community, the genotype defines the representation of a solution. A solution is defined as the phenotype.

and constitutes an essential step in designing a metaheuristic. The encoding must be suitable and relevant to the tackled optimization problem. Moreover, the efficiency of a representation is also related to the search operators applied on this representation (neighborhood, recombination, etc.). In fact, when defining a representation, one has to bear in mind how the solution will be evaluated and how the search operators will operate.

Many alternative representations may exist for a given problem. A representation must have the following characteristics:

- **Completeness:** One of the main characteristics of a representation is its completeness; that is, all solutions associated with the problem must be represented.
- **Connexity:** The connexity characteristic is very important in designing any search algorithm. A search path must exist between any two solutions of the search space. Any solution of the search space, especially the global optimum solution, can be attained.
- **Efficiency:** The representation must be easy to manipulate by the search operators. The time and space complexities of the operators dealing with the representation must be reduced.

Many straightforward encodings may be applied for some traditional families of optimization problems (Fig. 1.16). There are some classical representations that

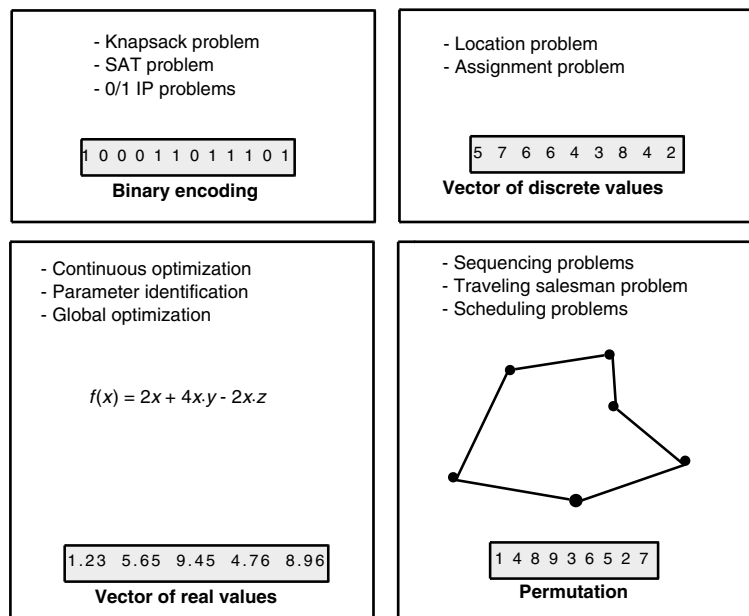


FIGURE 1.16 Some classical encodings: vector of binary values, vector of discrete values, vector of real values, and permutation.

are commonly used to solve a large variety of optimization problems. Those representations may be combined or underlying new representations. According to their structure, there are two main classes of representations: linear and nonlinear.

1.4.1.1 Linear Representations Linear representations may be viewed as strings of symbols of a given alphabet.

In many classical optimization problems, where the decision variables denote the presence or absence of an element or a yes/no decision, a *binary encoding* may be used. For instance, satisfiability problems and $\{0, 1\}$ -linear programs are representative of such problems. The binary encoding consists in associating a binary value for each decision variable. A solution will be encoded by a vector of binary variables.

Example 1.26 Binary encoding for knapsack problems. For a 0/1-knapsack problem of n objects, a vector s of binary variables of size n may be used to represent a solution:

$$\forall i, s_i = \begin{cases} 1 & \text{if object } i \text{ is in the knapsack} \\ 0 & \text{otherwise} \end{cases}$$

The binary encoding uses a binary alphabet consisting in two different symbols. It may be generalized to any *discrete values* based encoding using an n -ary alphabet. In this case, each variable takes its value over an n -ary alphabet. The encoding will be a vector of discrete values. This encoding may be used for problems where the variables can take a finite number of values, such as combinatorial optimization problems.

Example 1.27 Discrete encoding for generalized assignment problems. Many real-life optimization problems such as resource allocation may be reduced to assignment problems. Suppose a set of k tasks is to be assigned to m agents to maximize the total profit. A task can be assigned to any agent. A classical encoding for this class of problems may be based on a discrete vector s of size k , where $s[i]$ represents the agent assigned to the task i .

$$s[i] = j \quad \text{if the agent } j \text{ is assigned to task } i$$

Many sequencing, planning, and routing problems are considered as *permutation* problems. A solution for permutation problems, such as the traveling salesman problem and the permutation flow-shop scheduling problem, may be represented by a permutation $\pi = (\pi_1, \pi_2, \dots, \pi_n)$. Any element of the problem (cities for the TSP, jobs for the FSP) must appear only once in the representation.

Example 1.28 Permutation encoding for the traveling salesman problem. For a TSP problem with n cities, a tour may be represented by a permutation of size n . Each permutation decodes a unique solution. The solution space is represented by the set of all permutations. Its size is $|S| = (n - 1)!$ if the first city of the tour is fixed.

Single-Solution Based Metaheuristics

While solving optimization problems, single-solution based metaheuristics (S-metaheuristics) improve a single solution. They could be viewed as “walks” through neighborhoods or search trajectories through the search space of the problem at hand [163]. The walks (or trajectories) are performed by iterative procedures that move from the current solution to another one in the search space. S-metaheuristics show their efficiency in tackling various optimization problems in different domains.

In this chapter, a unified view of the common search concepts for this class of metaheuristics is presented. Then, the main existing algorithms, both from the design and the implementation point of view, are detailed in an incremental way.

This chapter is organized as follows. After the high-level template of S-metaheuristics is presented, Section 2.1 details the common search components for S-metaheuristics: the definition of the neighborhood structure, the incremental evaluation function, and the determination of the initial solution. The concept of very large neighborhoods is also presented. Section 2.2 discusses the landscape analysis of optimization problems. Sections 2.3, 2.4, and 2.5 introduce, in an incremental manner, the well-known S-metaheuristics: local search, simulated annealing, and tabu search. Then, Sections 2.6, 2.7, and 2.8 present, respectively, the iterative local search, the variable neighborhood search, and the guided local search algorithms. In Section 2.9, other S-metaheuristics are dealt with, such as GRASP, the noisy and the smoothing methods. Finally, Section 2.10 presents the ParadisEO–MO (moving objects) module of the ParadisEO framework that is dedicated to the implementation of S-metaheuristics. Some design and implementations of S-metaheuristics such as local search, simulated annealing, tabu search, and iterated local search are illustrated.

2.1 COMMON CONCEPTS FOR SINGLE-SOLUTION BASED METAHEURISTICS

S-metaheuristics iteratively apply the generation and replacement procedures from the current single solution (Fig. 2.1). In the generation phase, a set of candidate solutions are generated from the current solution s . This set $C(s)$ is generally obtained by local

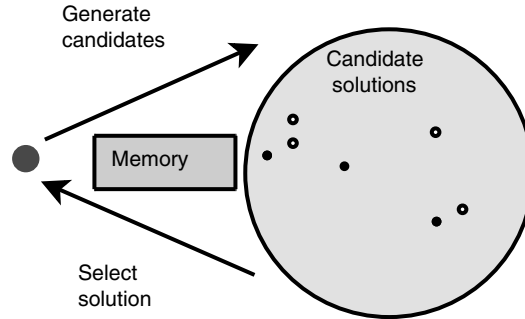


FIGURE 2.1 Main principles of single-based metaheuristics.

transformations of the solution. In the replacement phase,¹ a selection is performed from the candidate solution set $C(s)$ to replace the current solution; that is, a solution $s' \in C(s)$ is selected to be the new solution. This process iterates until a given stopping criteria. The generation and the replacement phases may be *memoryless*. In this case, the two procedures are based only on the current solution. Otherwise, some history of the search stored in a memory can be used in the generation of the candidate list of solutions and the selection of the new solution. Popular examples of such S-metaheuristics are local search, simulated annealing, and tabu search. Algorithm 2.1 illustrates the high-level template of S-metaheuristics.

Algorithm 2.1 High-level template of S-metaheuristics.

Input: Initial solution s_0 .
 $t = 0$;
Repeat
 /* Generate candidate solutions (partial or complete neighborhood) from s_t */
 Generate($C(s_t)$) ;
 /* Select a solution from $C(s)$ to replace the current solution s_t */
 $s_{t+1} = \text{Select}(C(s_t))$;
 $t = t + 1$;
Until Stopping criteria satisfied
Output: Best solution found.

The common search concepts for *all* S-metaheuristics are the definition of the *neighborhood* structure and the determination of the *initial solution*.

2.1.1 Neighborhood

The definition of the neighborhood is a required common step for the design of any S-metaheuristic. The neighborhood structure plays a crucial role in the performance

¹Also named transition rule, pivoting rule, and selection strategy.

of an S-metaheuristic. If the neighborhood structure is not adequate to the problem, any S-metaheuristic will fail to solve the problem.

Definition 2.1 Neighborhood. A neighborhood function N is a mapping $N : S \rightarrow 2^S$ that assigns to each solution s of S a set of solutions $N(s) \subset S$.

A solution s' in the neighborhood of s ($s' \in N(s)$) is called a *neighbor* of s . A neighbor is generated by the application of a *move* operator m that performs a small perturbation to the solution s . The main property that must characterize a neighborhood is *locality*. Locality is the effect on the solution when performing the move (perturbation) in the representation. When small changes are made in the representation, the solution must reveal small changes. In this case, the neighborhood is said to have a strong locality. Hence, a S-metaheuristic will perform a meaningful search in the landscape of the problem. Weak locality is characterized by a large effect on the solution when a small change is made in the representation. In the extreme case of weak locality, the search will converge toward a random search in the search space.

The structure of the neighborhood depends on the target optimization problem. It has been first defined in continuous optimization.

Definition 2.2 The neighborhood $N(s)$ of a solution s in a continuous space is the ball with center s and radius equal to ϵ with $\epsilon > 0$.

Hence, one have $N(s) = \{s' \in R^n / \|s' - s\| < \epsilon\}$. By using the Euclidean norm, we obtain $\|s' - s\| = \sqrt{(s'_1 - s_1)^2 + (s'_2 - s_2)^2 + \dots + (s'_n - s_n)^2}$, which is the Euclidean distance between s' and s (Fig. 2.2).

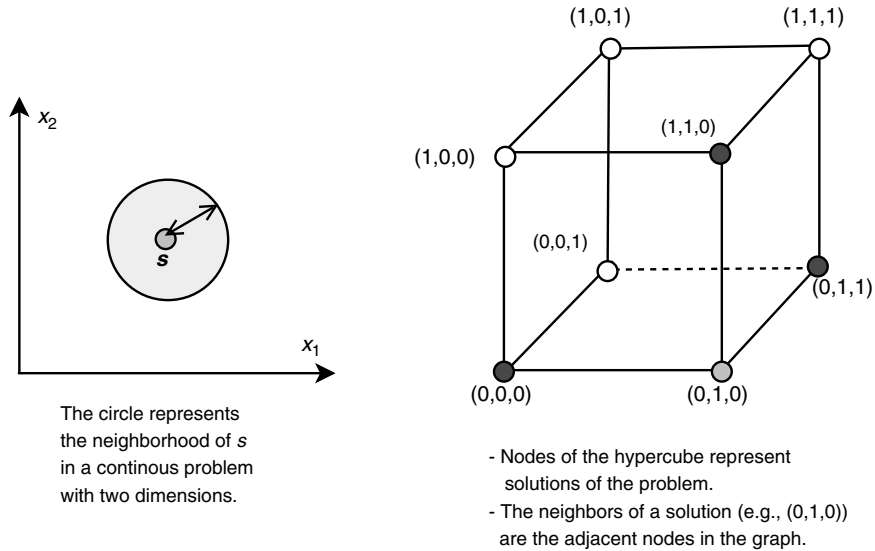


FIGURE 2.2 Neighborhoods for a continuous problem and a discrete binary problem.

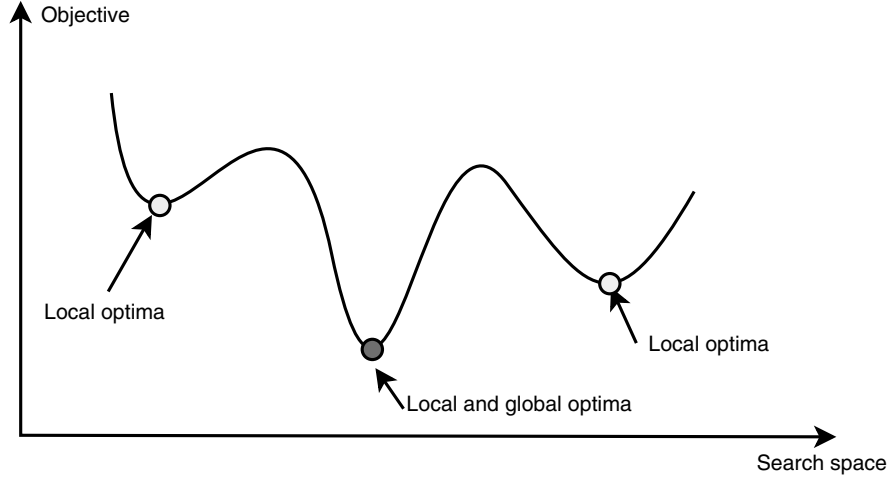


FIGURE 2.4 Local optimum and global optimum in a search space. A problem may have many global optimal solutions.

Definition 2.4 Local optimum. *Relatively to a given neighboring function N , a solution $s \in S$ is a local optimum if it has a better quality than all its neighbors; that is, $f(s) \leq f(s')^2$ for all $s' \in N(s)$ (Fig. 2.4).*

For the same optimization problem, a local optimum for a neighborhood N_1 may not be a local optimum for a different neighborhood N_2 .

Example 2.1 k -distance neighborhood versus k -exchange neighborhood. For permutation problems, such as the TSP, the exchange operator (swap operator) may be used (Fig. 2.5). The size of this neighborhood is $n(n-1)/2$ where n represents the number of cities. Another widely used operator is the k -opt operator,³ where k edges are removed from the solution and replaced with other k edges. Figure 2.5 (resp. Fig. 2.6) illustrates the application of the 2-opt operator (resp. 3-opt operator) on a tour. The neighborhood for the 2-opt operator is represented by all the permutations obtained by removing two edges. It replaces two directed edges (π_i, π_{i+1}) and (π_j, π_{j+1}) with (π_i, π_j) and (π_{i+1}, π_{j+1}) , where $i \neq j-1, j, j+1 \forall i, j$. Formally, the neighbor solution is defined as

$$\begin{aligned}\pi'(k) &= \pi(k) & \text{for } k \leq i \text{ or } k > j \\ \pi'(k) &= \pi(i+j+1-k) & \text{for } i < k \leq j\end{aligned}$$

The size of the neighborhood for the 2-opt operator is $[(n(n-1)/2) - n]$; all pairs of edges are concerned except the adjacent pairs.

²For a minimization problem.

³Also called k -exchange operator.

criterion. k_1 may be equal to $M \times D^2$, where D is the sum of the durations of all the operations of the instance. It is easy to see that the makespan of busy schedules is always smaller than this value. k_2 may be equal to $J \times M$, as there are $J \times M$ operations in the problem, so that there are no more than $J \times M$ critical operations.

Using the *same* metaheuristics, the new objective function f' provides better results in terms of the quality of the local optima that are found [232]. For this problem, this approach does not require a lot of implementation work and the computational cost is low.

The landscape analysis has some limitations. For instance, some measures require data that are difficult to obtain for complex real-life problems: global optimal solutions, number of local optima, distance definition, all solutions of distance d , and so on. However, approximations may be used to deal with those difficulties. As those approximations may lead to incorrect predictions, careful analysis must be carried out from those data [27,575]. Landscape analysis remains an interesting, valuable, and unavoidable tool for the analysis of the different search components of metaheuristics in solving complex optimization problems. The use of a set of different indicators is encouraged, rather than a single magic wand, to analyze the landscape of a problem and its difficulty.

2.3 LOCAL SEARCH

Local search¹² is likely the oldest and simplest metaheuristic method [3,598]. It starts at a given initial solution. At each iteration, the heuristic replaces the current solution by a neighbor that improves the objective function (Fig. 2.21). The search

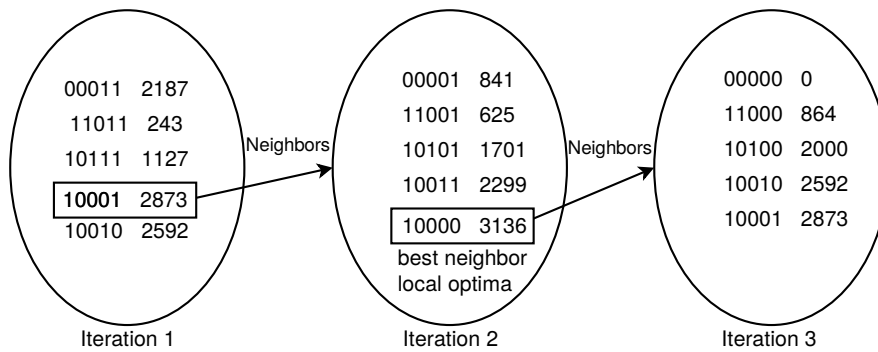


FIGURE 2.21 Local search process using a binary representation of solutions, a flip move operator, and the best neighbor selection strategy. The objective function to maximize is $x^3 - 60x^2 + 900x$. The global optimal solution is $f(01010) = f(10) = 4000$, while the final local optima found is $S = (10000)$, starting from the solution $S_0 = (10001)$.

¹²Also referred as hill climbing, descent, iterative improvement, and so on. In some literature, local search also refers to general S-metaheuristics.

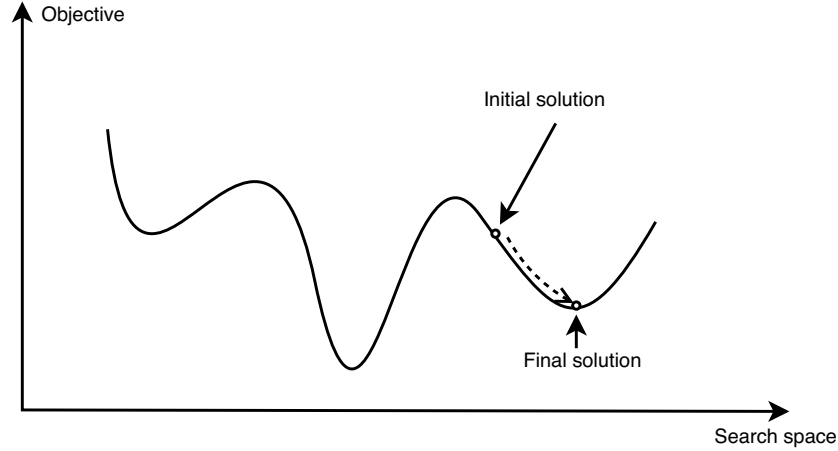


FIGURE 2.22 Local search (steepest descent) behavior in a given landscape.

stops when all candidate neighbors are worse than the current solution, meaning a local optimum is reached. For large neighborhoods, the candidate solutions may be a subset of the neighborhood. The main objective of this restricted neighborhood strategy is to speed up the search. Variants of LS may be distinguished according to the order in which the neighboring solutions are generated (deterministic/stochastic) and the selection strategy (selection of the neighboring solution) (Fig. 2.22).

LS may be seen as a descent walk in the graph representing the search space. This graph may be defined by $G = (S, V)$, where S represents the set of all feasible solutions of the search space and V represents the neighborhood relation. In the graph G , an edge (i, j) will connect to any neighboring solutions s_i and s_j . For a given solution s , the number of associated edges will be $|N(s)|$ (number of neighbors). Algorithm 2.2 illustrates the template of a local search algorithm.

Algorithm 2.2 Template of a local search algorithm.

```

 $s = s_0$  ; /* Generate an initial solution  $s_0$  */
While not Termination_Criterion Do
    Generate ( $N(s)$ ) ; /* Generation of candidate neighbors */
    If there is no better neighbor Then Stop ;
     $s = s'$  ; /* Select a better neighbor  $s' \in N(s)$  */
Endwhile
Output Final solution found (local optima).

```

From an initial solution s_0 , the algorithm will generate a sequence $s_1, s_2, \dots, s_k$ of solutions with the following characteristics (Fig. 1.32):

- The size of the sequence k is unknown *a priori*.
- $s_{i+1} \in N(s_i), \forall i \in [0, k - 1]$.

- $f(s_{i+1}) < f(s_i), \forall i \in [0, k - 1]$.¹³
- s_k is a local optimum: $f(s_k) \leq f(s), \forall s \in N(s_k)$.

In addition to the definition of the initial solution and the neighborhood, designing a local search algorithm has to address the selection strategy of the neighbor that will determine the next current solution.

2.3.1 Selection of the Neighbor

Many strategies can be applied in the selection of a better neighbor (Fig. 2.23):

- **Best improvement (steepest descent):** In this strategy, the best neighbor (i.e., neighbor that improves the most the cost function) is selected. The neighborhood is evaluated in a fully deterministic manner. Hence, the exploration of the neighborhood is *exhaustive*, and all possible moves are tried for a solution to select the best neighboring solution. This type of exploration may be time-consuming for large neighborhoods.
- **First improvement:** This strategy consists in choosing the first improving neighbor that is better than the current solution. Then, an improving neighbor is immediately selected to replace the current solution. This strategy involves a partial evaluation of the neighborhood. In a *cyclic* exploration, the neighborhood is evaluated in a deterministic way following a given order of generating the neighbors. In the worst case (i.e., when no improvement is found), a complete evaluation of the neighborhood is performed.
- **Random selection:** In this strategy, a random selection is applied to those neighbors improving the current solution.

A compromise in terms of quality of solutions and search time may consist in using the first improvement strategy when the initial solution is randomly generated

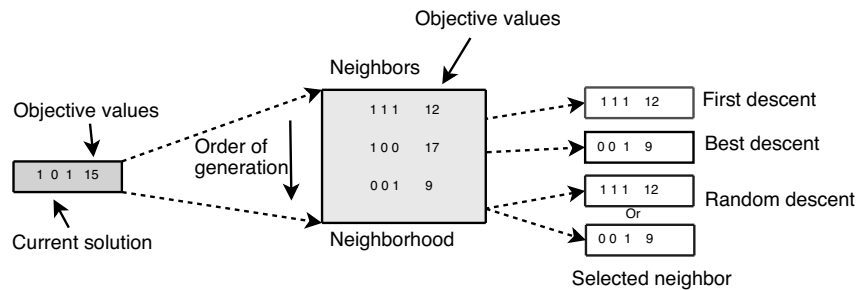


FIGURE 2.23 Selection strategies of improving neighbors.

¹³The problem is supposed to be a minimization problem.

and the best improvement strategy when the initial solution is generated using a greedy procedure. In practice, on many applications, it has been observed that the first improving strategy leads to the same quality of solutions as the best improving strategy while using a smaller computational time. Moreover, the probability of premature convergence to a local optima is less important in the first improvement strategy.

Example 2.21 Exhaustive versus cyclic exploration. Let us consider the neighborhood for the TSP that is based on the 2-opt move operator. A tour is represented by a permutation $(\pi_1, \pi_2, \dots, \pi_n)$. In the exhaustive exploration, all 2-opt moves are applied. In a cyclic exploration, the order may be given as follows: apply successfully the 2-opt operator using the exchange of vertices (π_1, π_2) with (π_3, π_4) , (π_4, π_5) , and so on until an improving neighbor is found. In the worst case, the whole neighborhood is explored.

The order of application can be changed during the search. For instance, the initial parameter that determines the order may be chosen randomly. Instead of using always the edge (π_1, π_2) , any edge (π_i, π_{i+1}) may represent the initial move.

Example 2.22 Cyclic exploration for a clique partitioning problem (CPP). Given a complete undirected graph $G = (V, E, w)$, where w represents positive or negative weights. To each edge (i, j) is associated a weight $w(i, j) = w(j, i)$. The objective of the CPP problem is to find a partition of the set of vertices V into k cliques $C_1, C_2, \dots, C_k$, minimizing the sum of the weights of the edges that have both end points in the same clique:

$$f(C_1, C_2, \dots, C_k) = \frac{1}{2} \sum_{l=1}^k \sum_{(i,j) \in C_l \times C_l, i \neq j} w(i, j)$$

The number of cliques k is not fixed [125].

A solution s for the CPP problem may be represented by a discrete vector, where each element of the vector specifies the class of the vertex. The size of the vector is equal to the number of vertices ($n = |V|$). The neighborhood may be defined by the transformation operator that consists in moving a single node to another class C . A class C of a solution s may be empty. The size of this neighborhood varies during the search and is equal to $(k+1)n - \alpha$, where k is the number of classes and α is the number of single-node classes.

The cyclic exploration of a neighborhood may be based on finding the best class for a given node i . Obviously, one can use the given order of the vertices $(v_1, v_2, \dots, v_n)$ or a random order generated uniformly. The vertices are explored in the specified order. A move is carried out as soon as an improving solution is found. All the nodes of the graph are tried once before the next time. This complete cyclic exploration is iterated until no improving neighbor is found. At the end of the process, all the nodes are in their best classes (i.e., local optimum related to the given move operator).